

# Algorithms for Data Science

## Exercise 1

(with Solutions)

*See the cheatsheet for valuable remarks!*

### Task 1.1: Coin Problem

You are given eight equally looking coins and a pair of scales. One of the coins is forged and has smaller weight, whereas all the other coins weigh the same. A single weighing on the scales consists of comparing the weight of any two (disjoint) subsets of the coins. Note that the weighting instrument does not show the actual weight.

- (a) Can you find the forged coin by using the scales three times?

**Solution:** Yes, e.g., by binary search.

- (b) Is it possible to find it using the scales only two times?

**Solution:** Yes, first compare two three-element sets. Then one weighing more is enough to determine the forged coin.

- (c) If we are allowed to do  $k$  weighings, what is the maximum number of coins among which we can find the forged coin?

**Solution:**  $3^k$ .

To come up with the formula, first find the exact maximum number of coins for small values of  $k$  ( $k = 0, 1, 2$ ). To do so, you have to (1) give a strategy for finding the forged coin and to (2) argue that if we have more than  $3^k$  coins, then it is impossible to find the forged one using the scales only  $k$  times.

| $k$ | maximum number of coins |
|-----|-------------------------|
| 0   | 1                       |
| 1   | 3                       |
| 2   | 9                       |

(For  $k = 0$  we use that there is exactly one forged coin.)

The task did not ask for a proof of the formula (since such a proof is a little out of the scope of our course), but for completeness we sketch the proof idea:

The strategy for general  $k$  is a generalization of the previous subtask: partition the  $3^k$  coins into three sets of equal size  $3^{k-1}$  and use the scales on any two of them. As the outcome, we know which of the three sets contains the forged coin. Proceed recursively on this set. After  $k$  steps, we arrive at a set of size  $3^{k-k} = 3^0 = 1$  and we are done.

Next we show that it is impossible to find the forged coin among more than  $3^k$  coins by using the scales only  $k$  times. Our proof uses the induction technique. Suppose we have shown

that for more than  $3^{k-1}$  coins, we need at least  $k$  weighings. (Indeed, we have already shown this claim for the smallest value(s) of  $k$ ; see the table above.) Consider a set with more than  $3^k$  coins. A weighing is nothing more than a partition of the coins into three sets and the information to which set the forged coin belongs to. At least one of the three sets contains more than  $3^k/3 = 3^{k-1}$  coins. In the worst case, it is the set containing the forged coin. By assumption, we need at least  $k$  more weighings to find the forged coin from this set. Thus, in the worst case, we need in total at least  $k + 1$  weighings if we have more than  $3^k$  coins.

**Task 1.2:** *Divide by Two*

In this task, we want to divide a given integer by two without using the division operation. Consider the following algorithm.

---

**Algorithm 1:**

---

**Input:** integer  $n \geq 0$   
**Output:** the value  $n/2$  (possibly fractional)  
 $\ell = 0$   
 $r = n$   
**while**  $\ell \neq r$  **do**  
     $\ell = \ell + 1$   
     $r = r - 1$   
**return**  $\ell$

---

- (a) How often does the while loop repeat in dependence of  $n$ ?

**Solution:** For even  $n$ :  $n/2$  times.  
For odd  $n$ : infinite loop!

- (b) The program is not correct. What is the error? Fix the mistake!

**Solution:**  
For odd  $n$ , the program does not terminate as  $\ell$  and  $r$  “jump over each other.”  
There are (at least) two different ways to fix the error:

1. Change  $\ell = \ell + 1$  to  $\ell = \ell + 0.5$  and  $r = r - 1$  to  $r = r - 0.5$ .

---

**Algorithm 2:**

---

**Input:** integer  $n \geq 0$   
**Output:** the value  $n/2$  (possibly fractional)  
 $\ell = 0$   
 $r = n$   
**while**  $\ell \neq r$  **do**  
     $\ell = \ell + 0.5$   
     $r = r - 0.5$   
**return**  $\ell$

---

2. Change the loop condition to  $\ell < r$ . After the loop, depending on whether  $n$  is even or not, return  $\ell$  or  $r + 0.5$ . (Note that if  $n$  is even, then  $\ell = r$ , otherwise  $r + 1 = \ell$ .)

---

**Algorithm 3:**

---

**Input:** integer  $n \geq 0$   
**Output:** the value  $n/2$  (possibly fractional)  
 $\ell = 0$   
 $r = n$   
**while**  $\ell < r$  **do**  
     $\ell = \ell + 1$   
     $r = r - 1$   
**if**  $\ell = r$  **then**  
    **return**  $\ell$   
**else** //  $r + 1 = \ell$   
    **return**  $r + 0.5$

---

- (c) Prove the correctness of the fixed algorithm with a loop invariant. Show also that the algorithm terminates.

**Solution:**

**Proof of Algorithm 2** Our algorithm terminates: after each iteration of the loop, the difference  $r - \ell$  gets smaller by 1. Initially the difference is the nonnegative integer  $n$ . Thus, after finitely many steps we have  $r - \ell = 0$  which is equivalent to  $\ell = r$  which means that the loop condition will be violated.

To prove correctness, we choose “ $n = \ell + r$ ” to be our loop invariant. (Intuitively, the statement says that “ $n/2$  is the middle point of the interval  $[\ell, r]$ .”)

The invariant is true:

- In the beginning of the first iteration (that is, the first time when the loop condition is checked by the program), we have  $\ell = 0$  and  $r = n$ , thus  $\ell + r = n$  and the invariant statement holds.
- Assume that in the beginning of some iteration (where the loop is not exited) the invariant statement holds for some values  $\ell'$  and  $r'$ . At the end of that iteration we have  $\ell = \ell' + 0.5$  and  $r = r' - 0.5$ . Thus  $\ell + r = \ell' + 0.5 + r' - 0.5 = \ell' + r'$  which is equal to  $n$  by assumption; hence, the invariant statement holds also for the next iteration.

The output of Algorithm 2 is correct (hence, the invariant is helpful): At the end, we exit the loop only if  $\ell = r$ . Together with our invariant, we have  $n = \ell + r = 2\ell$ , that is,  $n/2 = \ell$ ; our output is correct!

**Proof of Algorithm 3** Our algorithm terminates: after each iteration of the loop, the difference  $r - \ell$  gets smaller by 2; thus, after a finite number of iterations we have  $r - \ell \leq 0$ , that is,  $\ell \geq r$  which means that the loop condition will be violated.

To prove correctness, we choose the following statement as our loop invariant: “(i)  $n = \ell + r$ , (ii)  $\ell \leq r + 1$  and (iii)  $\ell, r$  are integers”. (Intuitively, fact (i) says that “ $n/2$  is the middle point of the interval  $[\min(\ell, r), \max(\ell, r)]$ .”)

The invariant is true:

- In the beginning of the first iteration (that is, the first time when the loop condition is checked by the program), we have  $\ell = 0$  and  $r = n$ , thus (i) holds as  $\ell + r = n$ , and (ii) and (iii) hold, as  $r = n$  is a nonnegative integer and  $\ell = 0 \leq r \leq r + 1$ ; hence, the invariant holds in the beginning.
- Assume that in the beginning of some iteration (where the loop is not exited) the invariant statement holds for some values  $\ell'$  and  $r'$ . At the end of that iteration, we have  $\ell = \ell' + 1$  and  $r = r' - 1$ . Thus,  $\ell$  and  $r$  are also integers (fact (iii) also holds) and given that the loop condition was satisfied with  $\ell' < r'$ , we have  $\ell \leq r + 1$  (fact (ii) holds). Fact (iii) also holds for  $\ell$  and  $r$  as  $\ell + r = \ell' + 1 + r' - 1 = \ell' + r'$  which is equal to  $n$  by assumption. Thus, the invariant holds also for the next iteration.

The output of Algorithm 3 is correct (hence, the invariant is helpful): At the end, we exit the loop when  $\ell \not\leq r$ , that is, when (iv)  $\ell \geq r$ . Assuming that our invariant is true, facts (ii), (iii) and (iv) imply that either  $\ell = r$  or  $r + 1 = \ell$ . If  $\ell = r$ , then fact (i) implies  $n = \ell + r = 2\ell$ , that is,  $n/2 = \ell$  and our output is correct for this case! In the other case of  $r + 1 = \ell$ , fact (i) implies  $n = \ell + r = 2r + 1$ , that is,  $n/2 = r + 1/2$  and, again, our output is correct!

## Bonus Tasks

Tasks for solving at home (not necessarily discussed in classes); relevant for short tests and the exam.

### Task 1.3: *Smallest Element*

In this task, we want to find a smallest element in an unordered array.

- (a) Consider the following algorithm.

---

**Algorithm 4:** Algorithm A

---

**Input:** array  $a[1..n]$

**Output:** the position of a smallest element

$minPos = -1$

**for**  $i = 1$  **to**  $n$  **do**

$isSmallest = \text{true}$

**for**  $j = 1$  **to**  $n$  **do**

**if**  $a[i] \geq a[j]$  **then**

$isSmallest = \text{false}$

**if**  $isSmallest$  **then**

$minPos = i$

**return**  $minPos$

---

- i. What is the output of the algorithm? Fix the error and discuss why the algorithm is now correct!

**Solution:** The output is always  $-1$  as the line  $minPos = i$  is never reached since for every  $i$ , when the inner loop reaches  $j = i$ , the condition  $a[i] \geq a[j]$  is obviously satisfied and thus  $isSmallest$  is set to false.

To fix this problem, change  $a[i] \geq a[j]$  to  $a[i] > a[j]$ . This is enough. Now  $isSmallest$  is set to false only if there exists a smaller value than  $a[i]$ , that is, if  $a[i]$  is not the smallest.

- ii. How often does the (repaired) algorithm compare two elements of  $a$  (that is,  $a[i] \geq a[j]$ ) in dependence of  $n$  in the worst case?

**Solution:**  $n^2$ .

- iii. Suppose that the running time depends only on comparing two elements of  $a$ . Further suppose that your computer can compare two elements of  $a$  in one millisecond. What is the worst case running time of the (fixed) algorithm for  $n = 1000\,000$  on your computer?

**Solution:** About 32 years.

- (b) Write a faster algorithm by filling the gaps of Algorithm B below.

---

**Algorithm 5:** Algorithm B

---

**Input:** array  $a[1..n]$   
**Output:** the position of a smallest element

```

1  $minPos = 1$ 
2  $j = 2$ 
3 while ..... do
4   .....
5   .....
6    $j = j + 1$ 
7 return  $minPos$ 
```

---

**Solution:**

---

**Algorithm 6:** Algorithm B

---

**Input:** array  $a[1..n]$   
**Output:** the position of a smallest element

```

1  $minPos = 1$ 
2  $j = 2$ 
3 while  $j \leq n$  do
4   if  $a[minPos] > a[j]$  then
5      $minPos = j$ 
6    $j = j + 1$ 
7 return  $minPos$ 
```

---

- i. How often does Algorithm B compare two elements of  $a$

**Solution:**  $n - 1$

- ii. Suppose that the running time depends only on comparing two elements of  $a$ . Further suppose that your computer can compare two elements of  $a$  in one millisecond. What is the worst case running time of the algorithm for  $n = 1000\ 000$  on your computer?

**Solution:** About 17 minutes.

- iii. Formulate an invariant depending on  $a$ ,  $j$  and  $minPos$ . Use the invariant to argue that Algorithm B is correct.

**Solution:** Invariant: “ $minPos$  is the position of the minimum element of the subarray  $a[1..j]$ .” It is true for  $j = 1$  and, assuming trueness for  $j$ , also true for  $j + 1$ . Thus, the invariant is true for all values of  $j$ , in particular for  $j = n$  when we leave the loop.

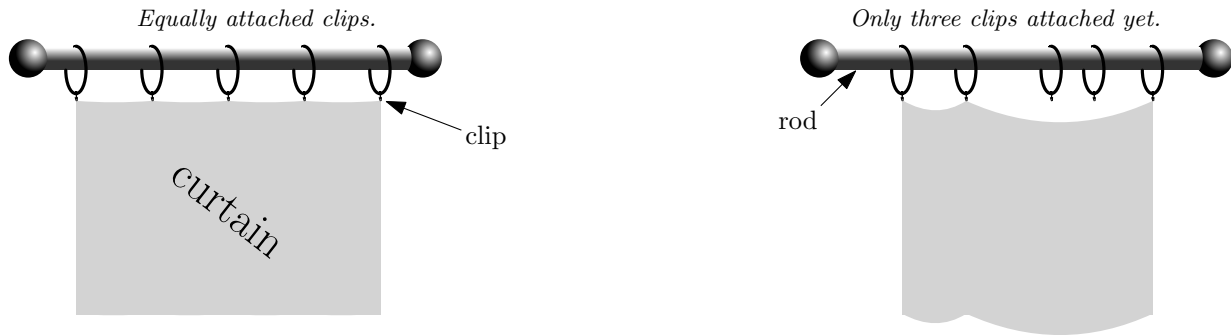
- (c) Discuss why any algorithm needs at least  $n - 1$  comparisons to find a minimum element in the worst case.

**Solution:** In the beginning, every element is a possible candidate for the minimum element. After each comparison, the set of candidates decreases by at most one element.

**Task 1.4:** *Curtain Problem - a real-world problem brought by the lecturer's wife*

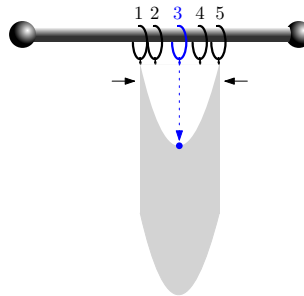
Suppose that you washed the curtain of your window and now you want to hang it up again. To do so, you have to attach the curtain to the five ring clips lying loosely on a rod above the window. It is important that the clips are equally spaced on the curtain, otherwise it won't look nice.

Unfortunately, you don't have any tool to measure the distances in order to find the right position for attaching the clips. However, you are a smart student and you want to use gravity and the idea underlying binary search to solve this problem.



- (a) Describe how to find equally spaced positions for the clips. Note that you can move the ring clips along the rod and that the curtain is obeying the laws of gravity. Your first step is to attach the top corners of the curtain to the left- and right-most clip.

**Solution:** After the two top corners are attached to the left- and right-most clip rings, we slide the two clips to each other. By gravity, the curtain folds into half and we can determine the position to attach the middle clip; see the figure. We proceed recursively.



- (b) Does your procedure also work for seven clips?

**Solution:** No, in the recursive step we have an even number of clips and therefore no unique middle clip.

- (c) For what numbers of clips does your procedure work?

**Solution:** For any number where there is a unique middle clip in each recursive step. Thus, for  $2 + (1 + 2 + \dots + 2^{i-1}) = 2 + 2^i - 1 = 2^i + 1$  clips.

**Task 1.5: *Guess my number!*—Level 2**

Consider the variant of the game *Guess my number!* when there is no upper bound on the integer number  $x$  to be guessed. We only know  $x \geq 8$ . Propose an algorithm to solve the problem using at most  $c \cdot \log_2 x$  questions where  $c$  can be any constant number that does not depend on  $x$ ; e.g.  $c = 8$ .

*Hint:* First, try to find an upper bound  $r$  of  $x$  with as few questions as possible. Then, run Binary Search from the lecture. You can assume that it needs only  $\log_2 n$  questions when  $n$  is the size of the given interval containing  $x$ .

**Solution:** We do exponential search to find  $r$  by testing  $r = 2^0, 2^1, \dots, 2^i$  (we could start from  $r = 8 = 2^3$ ). When we found such an  $r$ , then we have  $x \leq r < 2x$ . Recall that we choose  $r$  such that  $r = 2^i$  for some  $i$ . Thus, we needed at most  $i+1$  questions for finding  $r$ . Since we have  $i = \log_2 r$  and  $\log_2 r \leq \log_2(2x)$ , we can bound the number of questions for finding  $r$  by  $i+1 \leq \log_2(2x) + 1$ . As the interval length (of  $[3..r]$ ) is less than  $2x$ , we need at most  $\log_2 2x$  questions to find  $x$  by our assumption. Thus, the total number of questions is bounded by  $2\log_2(2x) + 1 = 2\log_2 x + 3 \leq 3\log_2 x$  (we use the rule  $\log_2(2x) = 1 + \log_2 x$  and that  $\log_2 x \geq 3$  as  $x \geq 8$ ).

**Task 1.6: *Real Time Running Times***

Given are five algorithms for the same (unknown) problem that get an array of size  $n$  as the input. The number of operations of the algorithms in dependence of  $n$  are  $\log_2 n$ ,  $n$ ,  $n \log_2 n$ ,  $n^2$  and  $2^n$ , respectively. Give their running times using an adequate time unit (e.g. nanoseconds, ..., years) in each case for different values of  $n$ . You can round the time values to integers and ignore running times above 10 years.

- (a) Suppose your computer can do 10 operations per second.

|                | $\log_2 n$ | $n$ | $n \log_2 n$ | $n^2$ | $2^n$ |
|----------------|------------|-----|--------------|-------|-------|
| n=10           |            |     |              |       |       |
| n=20           |            |     |              |       |       |
| n=21           |            |     |              |       |       |
| n=1000         |            |     |              |       |       |
| n=1000 000     |            |     |              |       |       |
| n=1000 000 000 |            |     |              |       |       |

**Solution:**

|                | $\log_2 n$ | $n$  | $n \log_2 n$ | $n^2$      | $2^n$                         |
|----------------|------------|------|--------------|------------|-------------------------------|
| n=10           | 300ms      | 1s   | 3s           | 10s        | 2min                          |
| n=20           | 400ms      | 2s   | 9s           | 40s        | 1d                            |
| n=21           | 400ms      | 2s   | 9s           | 44s        | 2d                            |
| n=1000         | 1s         | 2min | 17min        | 1d         | $> 3 \cdot 10^{292} \text{y}$ |
| n=1000 000     | 2s         | 1d   | 23d          | 3k y       |                               |
| n=1000 000 000 | 3s         | 3y   | 95y          | 3000 mio y |                               |



- (b) Suppose your computer can do 1000 000 operations per second.

|                | $\log_2 n$ | $n$ | $n \log_2 n$ | $n^2$ | $2^n$ |
|----------------|------------|-----|--------------|-------|-------|
| n=10           |            |     |              |       |       |
| n=20           |            |     |              |       |       |
| n=21           |            |     |              |       |       |
| n=1000         |            |     |              |       |       |
| n=1000 000     |            |     |              |       |       |
| n=1000 000 000 |            |     |              |       |       |

**Solution:**

|                | $\log_2 n$ | $n$       | $n \log_2 n$ | $n^2$      | $2^n$                 |
|----------------|------------|-----------|--------------|------------|-----------------------|
| n=10           | $3\mu s$   | $10\mu s$ | $33\mu s$    | $100\mu s$ | 1ms                   |
| n=20           | $4\mu s$   | $20\mu s$ | $86\mu s$    | $400\mu s$ | 1s                    |
| n=21           | $4\mu s$   | $21\mu s$ | $92\mu s$    | $441\mu s$ | 2s                    |
| n=1000         | $10\mu s$  | 1ms       | 10ms         | 1s         | $> 3 \cdot 10^{287}y$ |
| n=1000 000     | $20\mu s$  | 1s        | 20s          | 12d        |                       |
| n=1000 000 000 | $30\mu s$  | 17min     | 8h           | 32k y      |                       |

## Advanced Tasks

*Challenging tasks for motivated students, but beyond the scope of this course. Their difficulty level is comparable to that of standard computer science courses.*

### Task 1.7: Computing the Square Root

Write an algorithm that, given a nonnegative integer  $n$ , computes the value  $\lfloor \sqrt{n} \rfloor$ . The algorithm is not allowed to call other functions and is allowed to have only a single loop. This loop is not allowed to iterate more than  $\lceil \log_2 n \rceil + 1$  times. Prove the correctness of your algorithm.

**Solution:** Do binary search in the interval  $[1, n]$  to find an integer  $m$  such that  $m \cdot m \leq n < (m+1) \cdot (m+1)$ . Use appropriate loop invariants to show that your algorithm terminates and that the output is correct.